

CC_OpenCl.dll

Enterprise Agent-Ready Driver Documentation

VRAM Budgeting · Agent Contracts · Pipeline Recipes · API Usage
Stand: 2026-05-06

1. Zweck und Architektur

CC_OpenCl.dll ist ein Windows-x64-OpenCL-Treiber für GPU-beschleunigte Feld-, Agenten-, Mycelium-, SubQG-, Bio-Brain-, Quantum- und Enterprise-Algorithmus-Domänen. Diese Dokumentation ist so erweitert, dass Entwickler, KI-Agenten und autonome Codegeneratoren den Treiber sicher einsetzen können.

Der Treiber ist zustandsbehaftet: Hostprogramme initialisieren eine GPU, erzeugen OpenCL-Buffer, übertragen Daten, starten Domänenfunktionen und lesen Ergebnisse zurück. Große Zustände sollten möglichst im VRAM bleiben.

Domäne	Technisches Modell	Typische Funktionen
Lifecycle	GPU-Kontext, Queue, Programme	initialize_gpu, shutdown_gpu, finish_gpu
Memory	OpenCL-Buffer und Transfers	allocate_gpu_memory, write/read_host_to_gpu_blocking
Mycelium	Nährstoff-, Pheromon-, Mood- und Koloniefelder	subqg_init_mycel, step_pheromone_*, step_mycel_update
SubQG	acht gekoppelte Multifield-Kanäle	subqg_set_multifield_state, subqg_get_multifield_state
Resonance	gekoppelte Feldoszillatoren	execute_resonant_field_step_gpu
Scheduler	GPU-seitige aktive Liste	execute_energy_gated_scheduler_gpu
Morphogenesis	tabellarische Zelltyp-Regeln	execute_morphogenetic_rule_step_gpu
Quantum	quantenähnliche Algorithmen	execute_grover_gpu, execute_vqe_gpu, execute_qaoa_gpu

2. Build und Einsatz

Empfohlener Build auf AMD/Windows im x64 Native Tools Command Prompt for VS 2022:

```
cd /d D:\CC_OpenCl_Driver_Enterprise
scripts\build_windows_msvc_amd_direct.bat
```

Build-Ergebnis: build\CC_OpenCl.dll, build\CC_OpenCl.lib und build\CC_OpenCl.exp.

3. Sichere Grundsequenz

```
initialize_gpu(gpu)
allocate required buffers
write initial data
execute domain functions
read result if needed
free buffers
shutdown_gpu(gpu)
```

Ein Agent darf keine Domänenfunktion vor initialize_gpu aufrufen und darf keine GPU-Handles nach free_gpu_memory wiederverwenden.

4. VRAM Budgeting

Enterprise-Default: $\text{planned_bytes} = \text{required_bytes} * 1.15$ und $\text{planned_bytes} \leq \text{CL_DEVICE_GLOBAL_MEM_SIZE} * 0.65$.

Domäne	Formel	Standardverbrauch
Mycelium Core	pheromone[N*C*K]*float + mood[N*C]*float + nutrient[N]*float + potential[N]*float + colonies[N]*uint8 + neighbors[N*K]*int	185.00 MiB
SubQG Multifield	8 scalar channels[N]*float	32.00 MiB
Resonance Field	state/velocity/drive/energy[N]*float + neighbors[N*K]*int + weights[N*K]*float	80.00 MiB
Energy Scheduler	active_flags[N]*uchar + active_indices[N]*int + active_count[1]*uint	5.00 MiB
Morphogenesis	cell_type[N]*uchar + nutrient/energy/potential[N]*float + rule	13.00 MiB

	tables[R]	
Bio-Brain Standard	neurons=N, fanout=K, synapses=N*K: neuron scalar buffers + sparse synapse tables	149.00 MiB
Quantum Statevector	complex amplitudes[2 ^q]*float2 + probability/phase scratch	256.00 MiB

Grid	N	Mycel MiB	SubQG MiB	Resonance MiB	Scheduler MiB	Morph MiB	Bio-Brain MiB
256x256	65,536	11.6	2.0	5.0	0.3	0.8	9.3
512x512	262,144	46.2	8.0	20.0	1.3	3.3	37.2
1024x1024	1,048,576	185.0	32.0	80.0	5.0	13.0	149.0
2048x2048	4,194,304	740.0	128.0	320.0	20.0	52.0	596.0
4096x4096	16,777,216	2960.0	512.0	1280.0	80.0	208.0	2384.0

5. Detail: Standard-Grid 1024 x 1024

Mycelium Core

Buffer	Typ	Elemente	Bytes/Element	Bytes	MiB
pheromone	float	N*C*K	4	134,217,728	128.00
mood	float	N*C	4	16,777,216	16.00
nutrient	float	N	4	4,194,304	4.00
potential	float	N	4	4,194,304	4.00
colonies	uint8	N	1	1,048,576	1.00
neighbors_sparse	int	N*K	4	33,554,432	32.00

Gesamt: 193,986,560 Bytes = 185.00 MiB.

SubQG Multifield

Buffer	Typ	Elemente	Bytes/Element	Bytes	MiB
energy	float	N	4	4,194,304	4.00
pressure	float	N	4	4,194,304	4.00
gravity	float	N	4	4,194,304	4.00
magnetism	float	N	4	4,194,304	4.00
temperature	float	N	4	4,194,304	4.00
potential	float	N	4	4,194,304	4.00
drift_x	float	N	4	4,194,304	4.00
drift_y	float	N	4	4,194,304	4.00

Gesamt: 33,554,432 Bytes = 32.00 MiB.

Resonance Field

Buffer	Typ	Elemente	Bytes/Element	Bytes	MiB
state	float	N	4	4,194,304	4.00
velocity	float	N	4	4,194,304	4.00
drive	float	N	4	4,194,304	4.00
energy	float	N	4	4,194,304	4.00
neighbors	int	N*K	4	33,554,432	32.00
weights	float	N*K	4	33,554,432	32.00

Gesamt: 83,886,080 Bytes = 80.00 MiB.

Energy Scheduler

Buffer	Typ	Elemente	Bytes/Element	Bytes	MiB
active_flags	uchar	N	1	1,048,576	1.00
active_indices	int	N	4	4,194,304	4.00
active_count	uint	1	4	4	0.00

Gesamt: 5,242,884 Bytes = 5.00 MiB.

Morphogenesis

Buffer	Typ	Elemente	Bytes/Element	Bytes	MiB
cell_type	uchar	N	1	1,048,576	1.00
nutrient	float	N	4	4,194,304	4.00
energy	float	N	4	4,194,304	4.00
potential	float	N	4	4,194,304	4.00
rule tables	mixed	R	20	320	0.00

Gesamt: 13,631,808 Bytes = 13.00 MiB.

Bio-Brain Standard

Buffer	Typ	Elemente	Bytes/Element	Bytes	MiB
neuron_v	float	N	4	4,194,304	4.00
neuron_u	float	N	4	4,194,304	4.00
neuron_input	float	N	4	4,194,304	4.00
neuron_threshold	float	N	4	4,194,304	4.00
neuron_plasticity	float	N	4	4,194,304	4.00
spikes	uint8	N	1	1,048,576	1.00
synapse_weight	float	N*K	4	33,554,432	32.00
synapse_src	int	N*K	4	33,554,432	32.00
synapse_dst	int	N*K	4	33,554,432	32.00
synapse_trace	float	N*K	4	33,554,432	32.00

Gesamt: 156,237,824 Bytes = 149.00 MiB.

Quantum Statevector

Buffer	Typ	Elemente	Bytes/Element	Bytes	MiB
amplitude_real_imag	float2	2^q	8	134,217,728	128.00
probability_scratch	float	2^q	4	67,108,864	64.00
phase_scratch	float	2^q	4	67,108,864	64.00

Gesamt: 268,435,456 Bytes = 256.00 MiB.

6. Enterprise API Contracts

execute_resonant_field_step_gpu

Führt einen gekoppelten Feldschritt mit Nachbarschaftsgradient, Trägheit, Drive, Dämpfung und Energieausgabe aus.

Buffer	Typ	Größe	Mutiert
state_buf	float	N	ja
velocity_buf	float	N	ja
drive_buf	float	N	nein
energy_buf	float	N	ja
neighbors_buf	int	N*K	nein
weights_buf	float	N*K	nein

Parameter	Sicherer Bereich
K	4, 8, 16
dt	0.001 - 0.05
damping	0.01 - 0.20
coupling	0.05 - 0.50
inertia	0.80 - 0.98
clamp_abs	>0 empfohlen

execute_energy_gated_scheduler_gpu

Extrahiert aktive Zellen/Agenten über ein Energiefeld und schreibt active_flags, active_indices und active_count. active_count_buf muss vor jedem Aufruf auf 0 gesetzt werden.

Buffer	Typ	Größe	Mutiert
energy_buf	float	N	ja
nutrient_buf	float	N	ja
active_flags_buf	uchar	N	ja
active_indices_buf	int	N	ja
active_count_buf	uint	1	ja

execute_morphogenetic_rule_step_gpu

Transformiert cell_type[N] anhand tabellarischer Regeln. Die erste passende Regel gewinnt; Regelreihenfolge ist Teil der Simulation.

Buffer	Typ	Größe	Mutiert
cell_type_buf	uchar	N	ja
nutrient_buf	float	N	ja
energy_buf	float	N	nein
potential_buf	float	N	ja
rule_in_type_buf	int	R	nein
rule_min_nutrient_buf	float	R	nein
rule_min_energy_buf	float	R	nein
rule_out_type_buf	int	R	nein
rule_delta_potential_buf	float	R	nein

7. Agent Function Selection

Ziel	Funktion
gekoppelte Feldaktivität	execute_resonant_field_step_gpu
aktive Teilmenge finden	execute_energy_gated_scheduler_gpu
Zelltypregeln anwenden	execute_morphogenetic_rule_step_gpu
Mycelium/Pheromone	step_pheromone *, step_mycel update
Multifeld lesen/schreiben	subqg_set_multifield_state, subqg_get_multifield_state
Quantum-Optimierung	Quantum-Funktionsgruppe
GPU-Speicher	Memory-Funktionsgruppe

8. Fehlerbehandlung

Fehler	Ursache	Agentenreaktion
CL_OUT_OF_RESOURCES	VRAM/Register/Workgroup erschöpft	N halbieren, K/C reduzieren, Domänen sequenziell
CL_MEM_OBJECT_ALLOCATION_FAILURE	Buffer zu groß	Einzelbuffer prüfen
CL_INVALID_MEM_OBJECT	falsches/gelöschtes Handle	Buffer neu erzeugen
CL_INVALID_KERNEL_ARGS	Vertragsbruch	Signatur und Buffergrößen prüfen
NaN/Inf	numerische Instabilität	clamp setzen, dt/coupling reduzieren
active_count == N	Scheduler bringt keinen Vorteil	Threshold erhöhen oder Full-Step
active_count == 0	keine Aktivität	Drive erhöhen oder Threshold senken

9. Pipeline Recipes

Resonance + Scheduler:

```

initialize_gpu
allocate state, velocity, drive, energy, neighbors, weights
allocate active_flags, active_indices, active_count
loop:
    execute_resonant_field_step_gpu
    write active_count = 0
    execute_energy_gated_scheduler_gpu
    read active_count
    adapt threshold or process active indices

```

10. Maschinenlesbare Nutzung

docs/DRIVER_MANIFEST.json enthält Treibername, Plattform, Buildsript, Domänen, VRAM-Formeln, Bufferverträge, sichere Parameterbereiche, Exportfunktionsliste und Ressourcenpolitik. Agentensysteme sollten zuerst dieses Manifest lesen.

11. Harte Regeln für KI-Agenten

1. Vor jeder Allokation VRAM schätzen.
2. `planned_bytes <= global_mem * 0.65`.
3. `active_count_buf` vor Schedulerlauf auf 0 setzen.
4. Keine alten Handles nach Free verwenden.
5. Keine NaN/Inf-Ergebnisse akzeptieren.
6. Bei `CL_OUT_OF_RESOURCES` Problemgröße reduzieren.
7. Quantum-q nur nach neuer Speicherprüfung erhöhen.
8. Regelreihenfolgen dokumentieren und versionieren.

12. Dokumentationsdateien

Datei	Zweck
docs/VRAM_BUDGETING.md	Speicherformeln und Tabellen
docs/AGENT_USAGE_GUIDE.md	KI-/Agenten-Nutzung
docs/AI_AGENT_CONTRACTS.md	harte API-Verträge
docs/DRIVER_MANIFEST.json	maschinenlesbarer Vertrag
docs/PIPELINE_RECIPES.md	Ablaufmuster
docs/ERROR_HANDLING.md	Recovery-Strategien
docs/PARAMETER_SAFETY.md	Parameterprofile
examples/	Referenzintegration